
Rapport de TP — Module SIR

Application CRM

JAX-RS · JPA · HSQLDB · Angular

Auteurs : Yann Mendel Konan
Esaie Omiyale

Formation : Master MIAGE

Année : 2025 / 2026

13 mai 2026

Table des matières

1	Présentation du projet	2
1.1	Contexte et objectif	2
1.2	Description de l'application	2
1.3	Stack technique	2
1.4	Architecture globale	3
2	Modèle Objet Relationnel (MOR)	4
2.1	Diagramme de classes	4
2.2	Description des entités	5
2.2.1	Account — entité racine	5
2.2.2	Hiérarchie d'héritage	5
2.2.3	Relations bidirectionnelles	5
2.2.4	Clé composite — ClientGroupe	5
3	Diagramme de cas d'utilisation	6
3.1	Description des cas d'utilisation	7
4	Couche DAO	8
4.1	Architecture des DAOs	8
4.2	Récapitulatif des DAOs et types de requêtes	9
4.3	Exemples de requêtes	9
4.3.1	Requête JPQL — AccountDAO	9
4.3.2	Named Query — entité Client	9
4.3.3	Criteria Query — ClientDAO	9
5	API REST	11
5.1	Configuration du serveur	11
5.2	Tableau complet des endpoints	11
5.3	Structure d'une réponse API	12
5.4	Documentation OpenAPI	12
6	Configuration technique	13
6.1	Persistance JPA — HSQLDB	13
6.2	Gestion du CORS	13
6.3	Sérialisation JSON — Jackson	13
6.4	Structure du projet	14
7	Conformité aux consignes du TP	15
	Conclusion	16

Chapitre 1

Présentation du projet

1.1 Contexte et objectif

Ce projet a été réalisé dans le cadre du module **SIR** (Systèmes d'Information Répartis) de l'ISTIC. L'objectif est de concevoir une application web complète composée :

- d'un **backend** exposant une API REST en Java avec JAX-RS et JPA ;
- d'un **frontend** Angular consommant cette API.

1.2 Description de l'application

L'application est un **CRM** (*Customer Relationship Management*) permettant à des utilisateurs enregistrés de gérer leurs contacts clients, de les organiser en groupes et de leur envoyer des messages.

Les fonctionnalités principales sont :

- **Gestion des comptes** : création de comptes Admin, User, Moral ou Physique.
- **Gestion des clients** : CRUD complet, recherche par email, pays ou sexe.
- **Gestion des groupes** : regroupement de clients, ajout/retrait de membres.
- **Messagerie** : envoi de messages à un client individuel ou à un groupe.
- **Tableau de bord** : statistiques par utilisateur.

1.3 Stack technique

Couche	Technologies
Backend	Java 17, JAX-RS (RESTEasy), JPA (Hibernate), HSQLDB
Serveur	Undertow (embarqué, port 8080)
API Doc	OpenAPI 3 / Swagger UI
Frontend	Angular (Angular CLI), TypeScript, Bootstrap 5, faker.js
Build	Maven

1.4 Architecture globale



Chapitre 2

Modèle Objet Relationnel (MOR)

2.1 Diagramme de classes

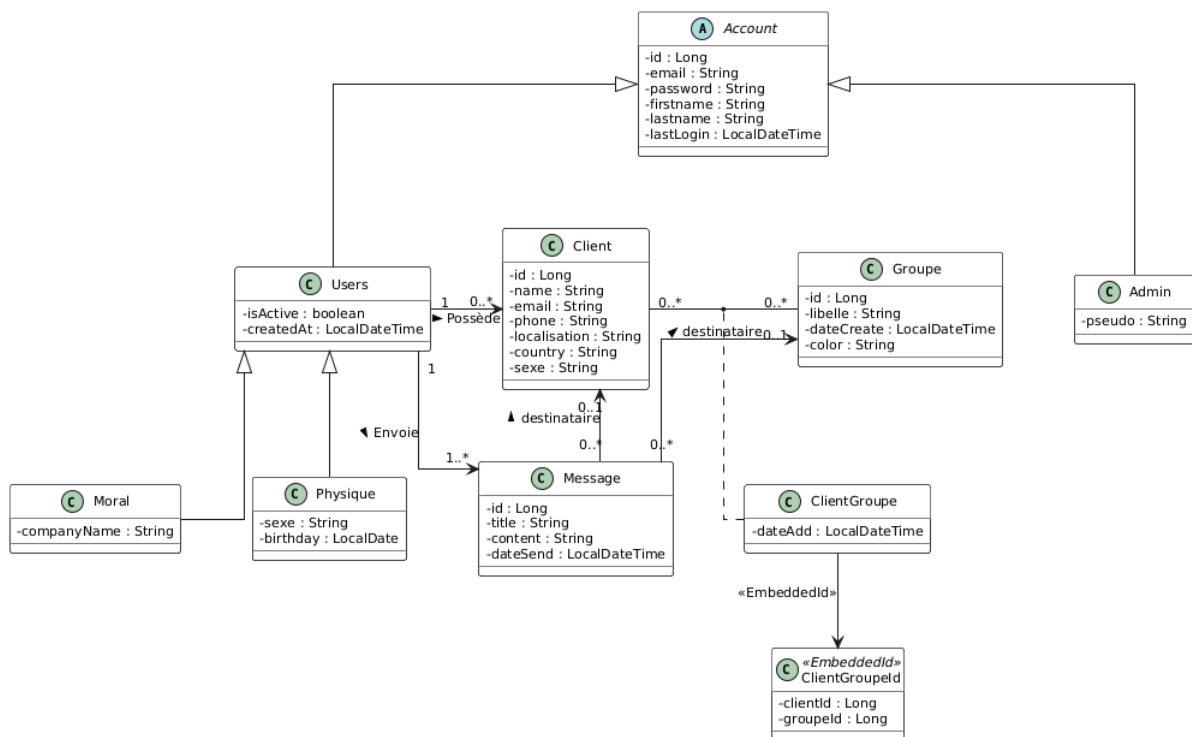


FIGURE 2.1 – Diagramme de classes — Application CRM

2.2 Description des entités

2.2.1 Account — entité racine

Account est la classe parente de toute la hiérarchie. Elle utilise la stratégie **SINGLE_TABLE** avec une colonne discriminante `type_account`.

Listing 2.1 – Héritage SINGLE_TABLE

```

1 @Entity
2 @Inheritance(strategy = InheritanceType.SINGLE_TABLE)
3 @DiscriminatorColumn(name = "type_account")
4 public class Account implements Serializable { ... }
```

2.2.2 Hiérarchie d'héritage

Classe	Discriminant	Attributs spécifiques
Admin	ADMIN	pseudo
Users	USER	isActive, createdAt
Moral	MORAL	companyName (hérite de Users)
Physique	PHYSIQUE	sexe, birthday (hérite de Users)

2.2.3 Relations bidirectionnelles

- **Users** ↔ **Client** : `Users.clients` / `Client.user`
- **Users** ↔ **Groupe** : `Users.groupes` / `Groupe.user`
- **Users** ↔ **Message** : `Users.messages` / `Message.sender`
- **Client** ↔ **ClientGroupe** : `Client.clientGroupes` / `ClientGroupe.client`
- **Groupe** ↔ **ClientGroupe** : `Groupe.clientGroupes` / `ClientGroupe.groupe`

2.2.4 Clé composite — ClientGroupe

Listing 2.2 – Clé primaire composite embarquée

```

1 @Embeddable
2 public class ClientGroupeId implements Serializable {
3     private Long clientId;
4     private Long groupeId;
5     // equals() et hashCode() obligatoires
6 }
7
8 @Entity
9 public class ClientGroupe implements Serializable {
10     @EmbeddedId
11     private ClientGroupeId id;
12     @ManyToOne @MapsId("clientId") private Client client;
13     @ManyToOne @MapsId("groupeId") private Groupe groupe;
14     private LocalDateTime dateAdd;
15 }
```

Chapitre 3

Diagramme de cas d'utilisation

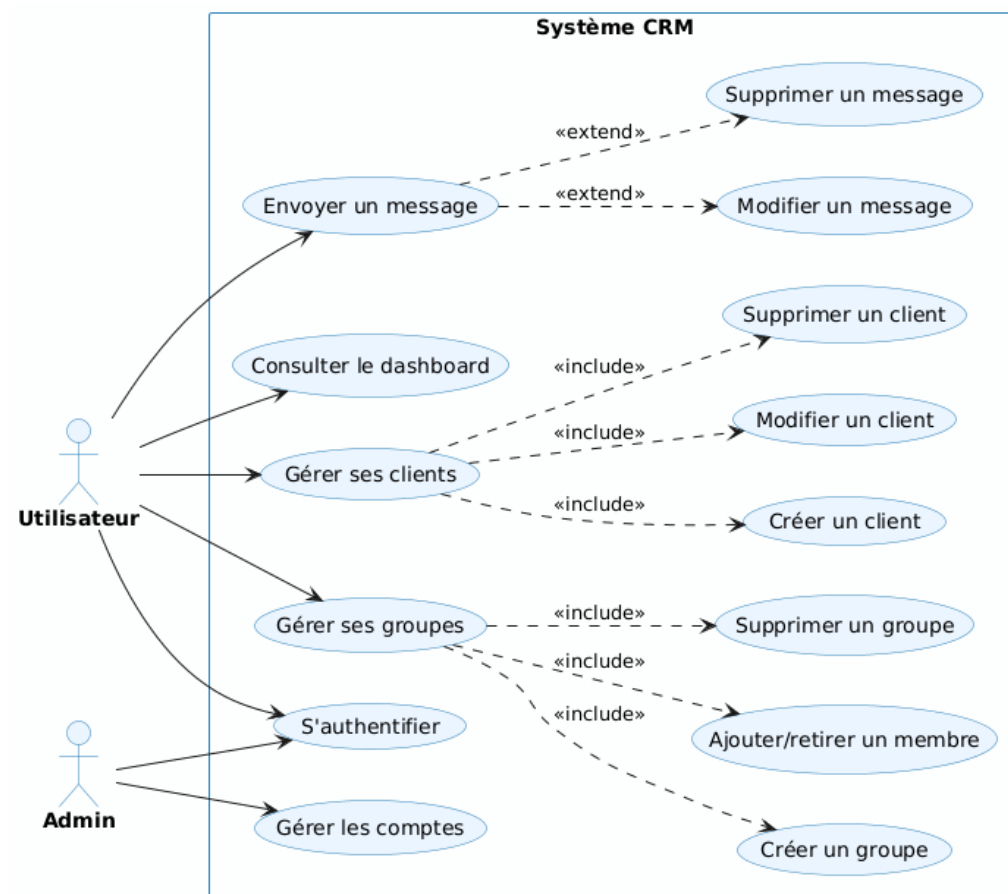


FIGURE 3.1 – Diagramme de cas d'utilisation — Application CRM

3.1 Description des cas d'utilisation

Cas d'utilisation	Acteur	Description
S'authentifier	Tous	Connexion via POST /accounts/login
Gérer ses clients	User	Créer, modifier, supprimer, rechercher des clients
Gérer ses groupes	User	Créer des groupes, ajouter/retirer des membres
Envoyer un message	User	Message vers un client ou un groupe entier
Consulter le dashboard	User	Statistiques personnelles
Gérer les comptes	Admin	CRUD sur tous les comptes, filtrer par type

Chapitre 4

Couche DAO

4.1 Architecture des DAOs

Toutes les DAOs héritent de `AbstractJpaDao<K,T>` qui fournit les opérations CRUD génériques. L'`EntityManager` est géré via un `ThreadLocal` dans `EntityManagerHelper`.

Listing 4.1 – `AbstractJpaDao` — patron générique

```
1 public abstract class AbstractJpaDao<K, T extends Serializable> {  
2     protected EntityManager entityManager;  
3     public AbstractJpaDao() {  
4         this.entityManager = EntityManagerHelper.getEntityManager();  
5     }  
6     public T findOne(K id) { return entityManager.find(clazz, id); }  
7     public List<T> findAll() { ... }  
8     public void save(T entity) { /* begin/persist/commit */ }  
9     public T update(T entity) { /* begin/merge/commit */ }  
10    public void delete(T entity) { /* begin/remove/commit */ }  
11 }
```

4.2 Récapitulatif des DAOs et types de requêtes

DAO	Types de requêtes	Méthodes métier notables
AccountDAO	JPQL	findByEmail, existsByEmail, findActiveUsers, findAllAdmins, findAllMorals, findAllPhysiques
ClientDAO	Named Query, Criteria Query, JPQL	findByEmail, findByCountry, findByCriteria(country, sexe), findByGroupe, countByUserId
GroupeDAO	Named Query, JPQL	findByLibelle, findByUserId, countByUserId, countMembersByGroupeId
MessageDAO	Named Query, JPQL	findBySender, deleteByGroupe, deleteByClient, countSentByUserId

4.3 Exemples de requêtes

4.3.1 Requête JPQL — AccountDAO

Listing 4.2 – Recherche par email (JPQL)

```

1 public Account findByEmail(String email) {
2     return entityManager.createQuery(
3         "SELECT a FROM Account a WHERE a.email = :email",
4         Account.class)
5         .setParameter("email", email)
6         .getResultList().stream().findFirst().orElse(null);
7 }

```

4.3.2 Named Query — entité Client

Listing 4.3 – Named Query déclarée sur l'entité

```

1 @NamedQuery(
2     name = "Client.findByCountry",
3     query = "SELECT c FROM Client c WHERE c.country = :country"
4 )

```

4.3.3 Criteria Query — ClientDAO

Listing 4.4 – Recherche multi-critères dynamique

```

1 public List<Client> findByCriteria(String country, String sexe) {
2     CriteriaBuilder cb = entityManager.getCriteriaBuilder();
3     CriteriaQuery<Client> cq = cb.createQuery(Client.class);
4     Root<Client> root = cq.from(Client.class);
5     List<Predicate> predicates = new ArrayList<>();
6     if (country != null)
7         predicates.add(cb.equal(root.get("country"), country));

```

```
8      if (sexe != null)
9          predicates.add(cb.equal(root.get("sexe"), sexe));
10     cq.where(predicates.toArray(new Predicate[0]));
11     cq.orderBy(cb.asc(root.get("name")));
12     return entityManager.createQuery(cq).getResultList();
13 }
```

Chapitre 5

API REST

5.1 Configuration du serveur

Le serveur est démarré via `RestServer.java` qui instancie un serveur **Undertow** embarqué sur le port **8080**.

- OpenAPI JSON : `http://localhost:8080/openapi.json`
- Swagger UI : `http://localhost:8080/swagger-ui/index.html`

5.2 Tableau complet des endpoints

Ressource	Méthode	Chemin	Description
Accounts	GET	<code>/accounts</code>	Lister / filtrer
	GET	<code>/accounts/{id}</code>	Récupérer par ID
	POST	<code>/accounts</code>	Créer
	PUT	<code>/accounts/{id}</code>	Mettre à jour
	DELETE	<code>/accounts/{id}</code>	Supprimer
	POST	<code>/accounts/login</code>	Authentification
Clients	GET	<code>/clients</code>	Lister / filtrer
	GET	<code>/clients/{id}</code>	Récupérer
	POST	<code>/clients</code>	Créer
	PUT	<code>/clients/{id}</code>	Mettre à jour
	DELETE	<code>/clients/{id}</code>	Supprimer
	GET	<code>/clients/by-user/{userId}</code>	Clients d'un utilisateur
	GET	<code>/clients/{id}/groupes</code>	Groupes d'un client
	POST	<code>/clients/{cId}/groupes/{gId}</code>	Ajouter à un groupe
Groupes	GET	<code>/groupes</code>	Lister
	GET	<code>/groupes/{id}</code>	Récupérer
	POST	<code>/groupes</code>	Créer
	PUT	<code>/groupes/{id}</code>	Mettre à jour
	DELETE	<code>/groupes/{id}</code>	Supprimer

(suite)

Ressource	Méthode	Chemin	Description
	GET	/groupes/by-user/{userId}	Groupes d'un utilisateur
	GET	/groupes/{id}/clients	Clients d'un groupe
	DELETE	/groupes/{gId}/clients/{cId}	Retirer un client
	GET	/groupes/{gId}/clients/not-in/{userId}	Clients d'un groupe
Messages	GET	/messages	Messages d'un client ou groupe
	GET	/messages/{id}	Récupérer par ID
	GET	/messages/sent/{senderId}	Messages envoyés
	POST	/messages	Envoyer
	PUT	/messages/{id}	Modifier
	DELETE	/messages/{id}	Supprimer
Dashboard	GET	/dashboard/{userId}	Statistiques utilisateur

5.3 Structure d'une réponse API

Listing 5.1 – Enveloppe ApiResponse

```

1 {
2   "status": 200,
3   "message": "OK",
4   "data": { ... }
5 }
```

5.4 Documentation OpenAPI

Listing 5.2 – Annotations OpenAPI sur AccountResource

```

1 @POST @Path("/login")
2 @Operation(summary = "Authentification",
3           description = "Verifie email + mot de passe.")
4 @ApiResponses({
5     @ApiResponse(responseCode = "200",
6                  description = "Authentification reussie"),
7     @ApiResponse(responseCode = "401",
8                  description = "Identifiants incorrects")
9 })
10 public Response login(AccountDTO dto) { ... }
```

Chapitre 6

Configuration technique

6.1 Persistence JPA — HSQLDB

Listing 6.1 – persistence.xml (HSQLDB)

```
<persistence-unit name="dev">
  <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
  <properties>
    <property name="jakarta.persistence.jdbc.driver"
      value="org.hsqldb.jdbcDriver"/>
    <property name="jakarta.persistence.jdbc.url"
      value="jdbc:hsqldb:hsql://localhost/">
    <property name="jakarta.persistence.jdbc.user" value="SA"/>
    <property name="jakarta.persistence.jdbc.password" value=""/>
    <property name="hibernate.hbm2ddl.auto" value="update"/>
    <property name="hibernate.dialect"
      value="org.hibernate.dialect.HSQLDialect"/>
    <property name="hibernate.show_sql" value="true"/>
  </properties>
</persistence-unit>
```

6.2 Gestion du CORS

La gestion du CORS est implémentée côté serveur Java, sans proxy Angular :

- CorsFilter (ContainerResponseFilter) : ajoute les en-têtes CORS à chaque réponse.
- CorsRequestFilter (ContainerRequestFilter) : intercepte les requêtes OPTIONS (preflight) et répond immédiatement avec 200.

L'origine autorisée est `http://localhost:4200`.

6.3 Sérialisation JSON — Jackson

JacksonConfig configure un ObjectMapper personnalisé :

- Support des types Java 8 Date/Time via `JavaTimeModule`.
- Désactivation de la sérialisation des dates en timestamps.

— Parseur LocalDateTime flexible : accepte HH:mm ou HH:mm:ss.

6.4 Structure du projet

```
src/main/java/fr/istic/taa/jaxrs/
|-- RestServer.java / TestApplication.java
|-- JacksonConfig.java / CorsFilter.java / CorsRequestFilter.java
|-- dao/
|   |-- AbstractJpaDao.java / EntityManagerHelper.java
|   '-- classic/ AccountDAO ClientDAO GroupeDAO MessageDAO
|-- entity/
|   |-- Account Admin Users Moral Physique
|   |-- Client Groupe Message
|   '-- ClientGroupe ClientGroupeId
|-- dto/
|   |-- AccountDTO ClientDTO GroupeDTO MessageDTO
|   '-- ClientGroupeDTO DashboardDTO ApiResponse
|-- service/
|   |-- AccountService ClientService GroupeService
|   '-- MessageService DashboardService
'-- rest/
    |-- AccountResource ClientResource GroupeResource
    '-- MessageResource DashboardResource
```

Chapitre 7

Conformité aux consignes du TP

Critère	Réalisé
JPA — Entités	
MOR fonctionnel (entités mappées et relations)	✓
Héritage SINGLE_TABLE (Admin, Users, Moral, Physique)	✓
Relation bidirectionnelle avec <code>mappedBy</code>	✓
DAO	
Une DAO par entité (Account, Client, Groupe, Message)	✓
Requête JPQL	✓
Requête nommée (<code>@NamedQuery</code>)	✓
Criteria Query	✓
Méthodes métier (au-delà du CRUD)	✓
API REST	
Un contrôleur par entité	✓
Documentation OpenAPI complète (<code>AccountResource</code>)	✓
Endpoints métiers (dashboard, recherche multicritère)	✓
Utilisation de DTOs	✓
Frontend	
Au moins un service GET	✓
Au moins un service POST / PUT	✓
Gestion CORS côté serveur (<code>CorsFilter</code> JAX-RS)	✓
Documentation	
Rapport avec modèle métier	✓
Diagramme de classes	✓
Diagramme de cas d'utilisation	✓
Description des endpoints	✓

Conclusion

Ce projet nous a permis de mettre en pratique les concepts fondamentaux des systèmes d'information répartis : modélisation objet avec JPA et héritage, exposition de services REST avec JAX-RS, transfert de données via DTOs et intégration avec un frontend Angular.

Les principales difficultés rencontrées ont été la gestion du cache JPA de niveau 1 (résolu par `em.clear()` et `em.refresh()`), la gestion des transactions imbriquées lors des suppressions en cascade, et la désérialisation du format `LocalDateTime` envoyé par Angular. Ces problèmes nous ont permis de mieux comprendre le fonctionnement interne de JPA et JAX-RS.

L'application CRM développée couvre l'ensemble des exigences du module : héritage JPA, relations bidirectionnelles, trois types de requêtes (JPQL, named queries, Criteria Query), documentation OpenAPI, et endpoints métiers dépassant le simple CRUD.